

Impedance Model

I Second order model representation

If we consider that a simple mass M , damper B , spring K , system is to be able to model the impedance behaviour of the human arm along an axis $\vec{x}$, while interacting with an environment then, we can write:

$$(Ms^2 + Bs + K)X_e(s) = F_e(s),$$

with X the endpoint position of the arm, and F the force of the arm on the environment, both along $\vec{x}$.

```
clear all
syms Ms Bs Ks;
% impedance parameters
Mv = 0.2;
Bv = 5;
Kv = 200;
```

The state representation of the system is:

$$\dot{X}(t) = \mathbf{A}X(t) + Bu(t),$$

$$y(t) = \mathbf{C}X(t)$$

$$X(t) = \begin{bmatrix} x_e(t) \\ \dot{x}_e(t) \end{bmatrix}, u(t) = f_e(t)$$

$$\mathbf{A} = \begin{bmatrix} 0 & 1 \\ -K_s/M_s & -B_s/M_s \end{bmatrix}$$

$$\mathbf{A} = \begin{pmatrix} 0 & 1 \\ -\frac{K_s}{M_s} & -\frac{B_s}{M_s} \end{pmatrix}$$

$$\mathbf{B} = [0; 1/M_s]$$

$$\mathbf{B} = \begin{pmatrix} 0 \\ \frac{1}{M_s} \end{pmatrix}$$

$$\mathbf{C} = [1, 0] \text{ \% or } \mathbf{C} = [0, 1] \text{ if the velocity is considered as output}$$

$$C = \begin{matrix} 1 \times 2 \\ 1 & 0 \end{matrix}$$

II Discretization

II-a State system

Assuming that the discretization is made through a zero order hold, and a sampling period T :

$$X[k+1] = A_d X[k] + B_d u[k]$$

$$A_d = e^{A T},$$

$$B_d = \left(\int_{\tau=0}^T e^{A\tau} d\tau \right) B = A^{-1}(A_d - I)B, \exists A^{-1}$$

```
dt = 1e-3; % 1ms
Ad = expm(A*dt) % ilaplace(inv(s*eye(2)-A),s,dt)
```

Ad =

$$\begin{pmatrix} \frac{\sigma_2 \sqrt{\sigma_3} - Bs \sigma_2 + Bs \sigma_1 + \sigma_1 \sqrt{\sigma_3}}{2 \sqrt{\sigma_3}} & -\frac{\sigma_1 \sigma_3 + Bs^2 \sigma_2 - \sigma_2 \sigma_3 - Bs^2 \sigma_1}{4 Ks \sqrt{\sigma_3}} \\ \frac{Ks \sigma_2 - Ks \sigma_1}{\sqrt{\sigma_3}} & \frac{Bs \sigma_2 + \sigma_2 \sqrt{\sigma_3} - Bs \sigma_1 + \sigma_1 \sqrt{\sigma_3}}{2 \sqrt{\sigma_3}} \end{pmatrix}$$

where

$$\sigma_1 = e^{-\frac{Bs - \sqrt{\sigma_3}}{2000 Ms}}$$

$$\sigma_2 = e^{-\frac{Bs + \sqrt{\sigma_3}}{2000 Ms}}$$

$$\sigma_3 = Bs^2 - 4 Ks Ms$$

```
Bd = A \ (Ad - eye(2)) * B % if A is not a singular matrix
```

Bd =

$$\begin{pmatrix} -\frac{e^{-\sigma_2} e^{-\sigma_1} (Bs^3 e^{\sigma_2} - Bs^3 e^{\sigma_1} - Bs e^{\sigma_2} \sigma_3 + Bs e^{\sigma_1} \sigma_3 + 2 Bs Ks Ms e^{\sigma_1} + 2 Ks Ms e^{\sigma_1} \sqrt{\sigma_3} - 2 Bs Ks Ms}{4 Ks^2 Ms \sqrt{\sigma_3}} \\ \frac{e^{-\sigma_2} e^{-\sigma_1} (e^{\sigma_1} \sigma_3 + Bs^2 e^{\sigma_2} - e^{\sigma_2} \sigma_3 - Bs^2 e^{\sigma_1})}{4 Ks Ms \sqrt{\sigma_3}} \end{pmatrix}$$

where

$$\sigma_1 = \frac{Bs - \sqrt{\sigma_3}}{2000 Ms}$$

$$\sigma_2 = \frac{Bs + \sqrt{\sigma_3}}{2000 Ms}$$

$$\sigma_3 = Bs^2 - 4 Ks Ms$$

$C_d = C;$

The eigenvalues λ of the eigenvector v of the discretized state system can be evaluated using either A or A_d :

$$\det(A - \lambda I)v = 0$$

```
eigAd = eig(Ad);
eigAd2 = exp(eig(A)*dt);
if (eigAd - eigAd2) ~= 0
    warning('The two eigen values vectors should be the same...')
end
```

However we can simply note that the characteristic polynomial of A is already known in this case, and therefore, the eigenvalues can be found by solving the roots of this polynomial:

$$M\lambda^2 + B\lambda + K = 0 \Leftrightarrow \lambda_{A_{1,2}} = \frac{-B \pm (B^2 - 4MK)^{\frac{1}{2}}}{2M}$$

Then we can simply deduced the eigenvalues of A_d : $\lambda_{A_d}^i = e^{\lambda_A^i T}$

$$\lambda_{A_d_{1,2}} = e^{\frac{-B \pm (B^2 - 4MK)^{\frac{1}{2}}}{2M} T}$$

II-b Transfer function form

We can now express the transfer function that relates the input force f_e to the output position x_e using

$H(z) = C_d (zI - A_d)^{-1} B_d$, with I the identity matrix.

```
z = sym('z');
```

```
Hd = simplify(Cd*(z*eye(2) - Ad)^(-1)*Bd);
[n,d] = numden(Hd);
n = collect(n,z)
```

n =

$$\left(2 e^{\frac{Bs}{1000 Ms}} \sigma_5 - \sigma_4 - \sigma_2 + \sigma_1 - \sigma_3\right) z + \sigma_2 - \sigma_4 + 2 \sigma_5 - \sigma_1 - \sigma_3$$

where

$$\sigma_1 = Bs e^{\frac{Bs - \sigma_5}{2000 Ms}}$$

$$\sigma_2 = Bs e^{\frac{Bs + \sigma_5}{2000 Ms}}$$

$$\sigma_3 = e^{\frac{Bs - \sigma_5}{2000 Ms}} \sigma_5$$

$$\sigma_4 = e^{\frac{Bs + \sigma_5}{2000 Ms}} \sigma_5$$

$$\sigma_5 = \sqrt{Bs^2 - 4 Ks Ms}$$

```
d = collect(d,z)
```

d =

$$\left(2 Ks e^{\frac{Bs}{1000 Ms}} \sigma_1\right) z^2 + \left(-2 \left(Ks e^{\frac{Bs + \sigma_1}{2000 Ms}} + Ks e^{\frac{Bs - \sigma_1}{2000 Ms}}\right) \sigma_1\right) z + 2 Ks \sigma_1$$

where

$$\sigma_1 = \sqrt{Bs^2 - 4 Ks Ms}$$

```
b_coef = flip(coeffs(n,z)); % high order first
a_coef = flip(coeffs(d,z));
```

We know now that we have a transfer function of the form:

$$\frac{X_e(z)}{F_e(z)} = H_d(z) = K_H \frac{z + b_0}{z^2 + a_1 z + a_0} = \frac{b_1' z + b_0'}{a_2' z^2 + a_1' z + a_0'}$$

$$K_H = \frac{b_1'}{a_2'}, b_0 = \frac{b_0'}{b_1'}, a_1 = \frac{a_1'}{a_2'}, a_0 = \frac{a_0'}{a_2'}$$

```
Kh = b_coef(1)/a_coef(1);
b0 = b_coef(2)/b_coef(1);
a1 = a_coef(2)/a_coef(1);
```

```
a0 = a_coef(3)/a_coef(1);
```

Knowing the desired/tested impedance parameters, it is already possible to evaluate the expected coefficient:

```
a1v = double(subs(a1,[Ks,Bs,Ms],[Kv,Bv,Mv]))
```

```
a1v = -1.9743
```

```
a0v = double(subs(a0,[Bs,Ms],[Bv,Mv])) % do not depends on K
```

```
a0v = 0.9753
```

```
Khv = double(subs(Kh,[Ks,Bs,Ms],[Kv,Bv,Mv]))
```

```
Khv = 2.4791e-06
```

```
Khb0v = double(subs(Kh*b0,[Ks,Bs,Ms],[Kv,Bv,Mv]))
```

```
Khb0v = 2.4585e-06
```

III Sensitivity analysis

The sensitivity analysis of the discrete coefficients to the impedance parameters affords a knowledge of the uncertainty of those coefficients according to the uncertainty of the parameters, and reversly. Therefore, it is possible to evaluate the robustness of the model to those uncertainties, unerlining the links between the inputs and outputs.

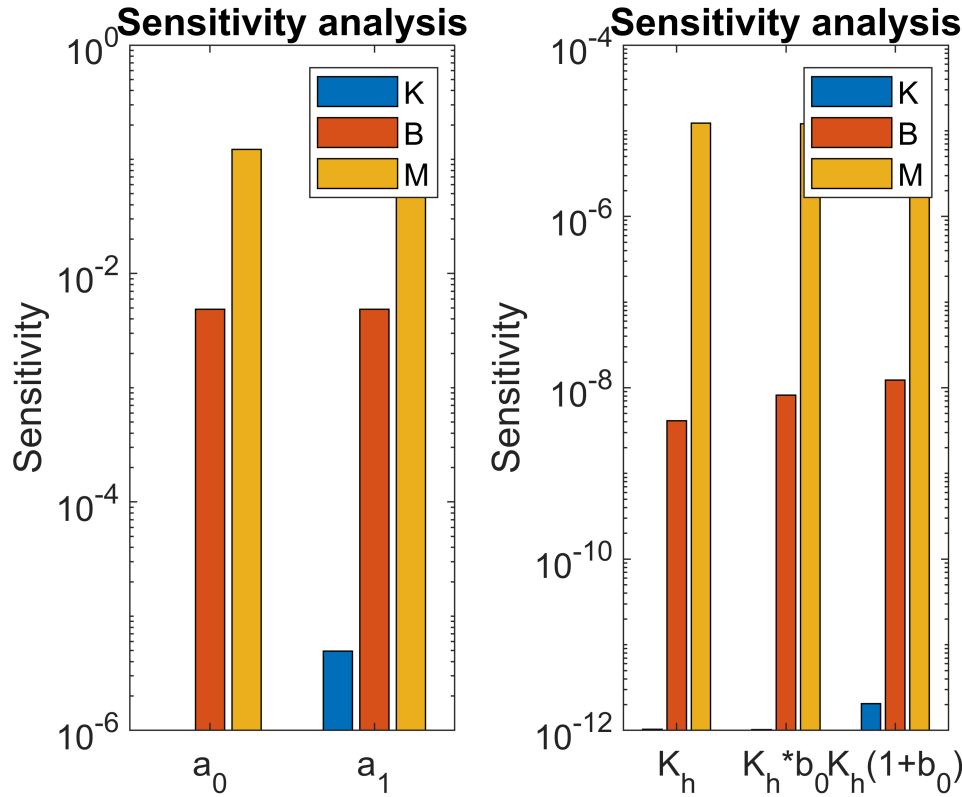
III-a Local method

Here we focus our attention on the influence of a single parameter at a time. To acheive that goal, we study the partial derivatives of every single parameter at an equilibrium point:

$$\frac{\partial y}{\partial x_i} \Big|_{x_0}$$

```
sens_a0 = double(subs(jacobian(a0,[Ks,Bs,Ms]),[Ks,Bs,Ms],[Kv,Bv,Mv]));
sens_a1 = double(subs(jacobian(a1,[Ks,Bs,Ms]),[Ks,Bs,Ms],[Kv,Bv,Mv]));
sens_Kh = double(subs(jacobian(Kh,[Ks,Bs,Ms]),[Ks,Bs,Ms],[Kv,Bv,Mv]));
sens_Khb0 = double(subs(jacobian(Kh*b0,[Ks,Bs,Ms]),[Ks,Bs,Ms],[Kv,Bv,Mv]));
sens_b = double(subs(jacobian(Kh*(1+b0),[Ks,Bs,Ms]),[Ks,Bs,Ms],[Kv,Bv,Mv]));
figure('DefaultAxesFontSize',14)
subplot(1,2,1)
bar(abs([sens_a0;sens_a1]))
set(gca,'YScale','log')
set(gca,'xticklabel',{'a_0';'a_1'})
title('Sensitivity analysis')
legend('K', 'B', 'M')
ylabel('Sensitivity')
subplot(1,2,2)
bar(abs([sens_Kh;sens_Khb0;sens_b]))
set(gca,'YScale','log')
set(gca,'xticklabel',{'K_h';'K_h*b_0';'K_h(1+b_0)'})
title('Sensitivity analysis')
```

```
legend('K', 'B', 'M')
ylabel('Sensitivity')
```



This local sensitivity analysis reveals that the parameters b_0 and K_h barely depends on the stiffness parameter K . As shown before, a_0 do not depends on K , and finally a_1 has a relatively low dependency on the stiffness compared to both the mass and the damping. We can observe 3 orders of difference (10^3) between the sensitivity of a_1 to K and B . From the inverse point of view, it means that knowing the coefficients, it will be difficult to estimate the stiffness.

IV Identification

The parameters a_1, a_0, b_0, K can be estimated using identification methodologies, with real dataset from human robot physical interactions.

IV-a Data generation

1) Real unperturbed data

```
load 'data_2020_Nov_17/data_without_impacts_2020_11_17.mat' 't' 'thetas' 'forces_unf'
addpath('utils') %
addpath('.././force_torque_sensor') % for force pre-processing
```

The data from the experiment is preprocessed to obtain a real shaped signal of the interaction force, that the robots applies on the environment $f_{r \rightarrow e}$.

```
exp_nb = randi([1 15],1,1); % chose a random experimental dataset
t = t{exp_nb} - t{exp_nb}(1); % t0 = 0s
f_tmp = forces_filtering(forces_unf{exp_nb}', forces_unf{exp_nb}', thetas{exp_nb}', t); % from
%
[b,a] = butter(2,50/(1/(2*dt)),'low'); % BW 2nd order low pass filter (cutoff freq. 50 Hz)
f_tmp = -f_tmp(3,:); % fz conversion from f(e->r) to f(r->e), robot force on the environment
f = filtfilt(b,a,f_tmp); % zero phase digital filtering
clear forces_unf thetas f_tmp % clear unnecessary data
```

2) Simulation of the perturbations

This real input is then injected in a simulated system with known impedance parameters:

```
input_force.signals.values = f;
input_force.time = t;
% perturbation introduced in simulation
ext_signal = 1;
pert_mag = 10;
pert_space = ceil(2.8/dt); % samples
pert_duration = 0.030/dt; % samples
% perturbation filter
xi = sqrt(2)/2;
w0 = 40*pi*1;
K_f = 3e3;
t_max = t(end);
out = sim('Impedance_env_simulation/KBM_sim',t_max);
```

The results from the simulation are collected:

```
fz = out.force.data;
z = out.position.data;
t = out.force.Time;
% get the rising edges indexes of the perturbations
pert_idx = find(diff(out.perturbations.data) > 0)';
pert_val = pert_mag.*ones(size(pert_idx));
```

3) Virtual trajectories estimation

The data is then processed to extract the differential force and position trajectories between the virtual and the actual trajectories:

```
% PARAMETERS
idx_wndw_virt_traj = ceil(0.200/dt); % 200ms (position)
idx_wndw_virt_f_traj = ceil(0.065/dt); % 65ms (force)
idx_wndw_imp_eval = ceil(0.200/dt); % 200ms
idx_delay = ceil(0.000/dt); % 0ms
idx_window = max(idx_wndw_imp_eval, idx_wndw_virt_traj);
nb_param = 3; % K B M
```

```
% DATA PRE-PROCESSING
```

```
delta_z = DIFF_TRAJECT(idx_window, idx_wndw_virt_traj, z, t, pert_idx, pert_val, idx_delay);  
delta_fz = DIFF_TRAJECT(idx_window, idx_wndw_virt_f_traj, fz, t, pert_idx, pert_val, idx_delay);
```

```
delta_z.computeDiffTraject('VirtTrajMethod', 'spline');
```

Warning: The perturbation n°114 was too close to the end of the experiment to be properly evaluated. It was therefore ignored, and so were the following ones.

```
delta_fz.computeDiffTraject('VirtTrajMethod', 'filterPlus'); % this step might take few seconds
```

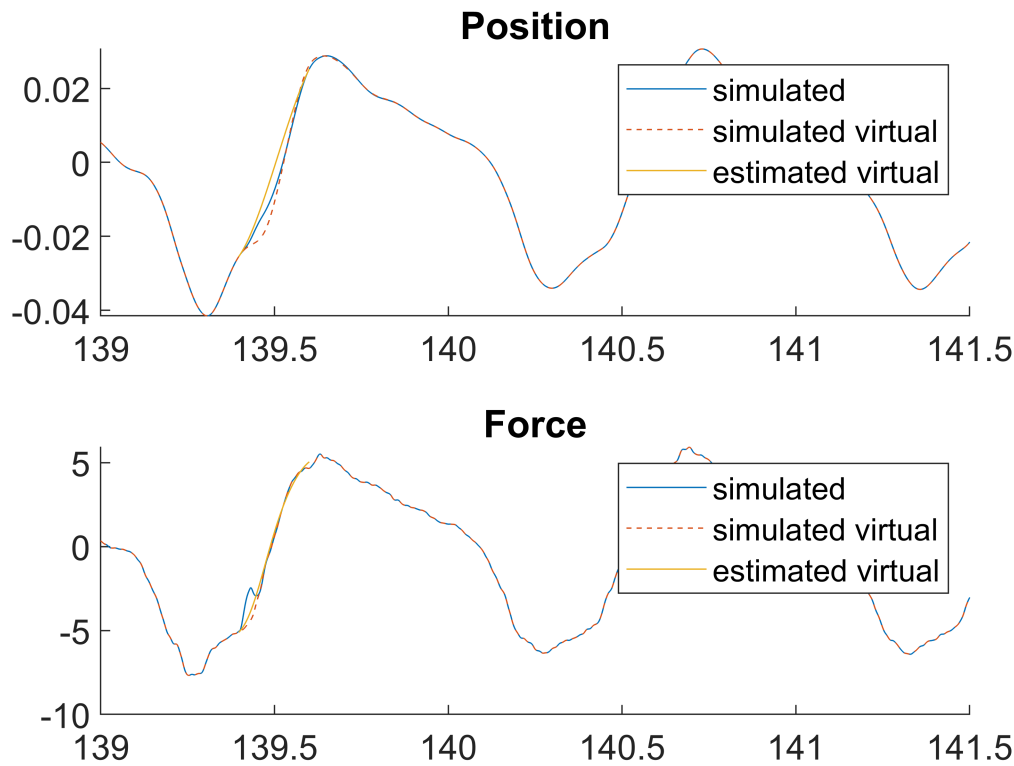
Warning: The perturbation n°114 was too close to the end of the experiment to be properly evaluated. It was therefore ignored, and so were the following ones.

```
delta_z.computeDerivatives();
```

4) Evaluation of the virtual trajectories

The produced methodology can be compared to the actual - *simulated* - virtual trajectories:

```
% real virtual trajectories  
fz0 = out.virt_force.data;  
z0 = out.virt_position.data;  
%  
figure('DefaultAxesFontSize',14)  
subplot(2,1,1)  
title('Position')  
hold on  
plot(t,z)  
plot(t,z0, '--')  
plot(delta_z.t_traject, delta_z.virt_traject, 'Color', [0.9290, 0.6940, 0.1250])  
xlim([139 141.5])  
legend('simulated', 'simulated virtual', 'estimated virtual')  
subplot(2,1,2)  
title('Force')  
hold on  
plot(t,fz)  
plot(t,fz0, '--')  
plot(delta_fz.t_traject, delta_fz.virt_traject, 'Color', [0.9290, 0.6940, 0.1250])  
xlim([139 141.5])  
legend('simulated', 'simulated virtual', 'estimated virtual')
```



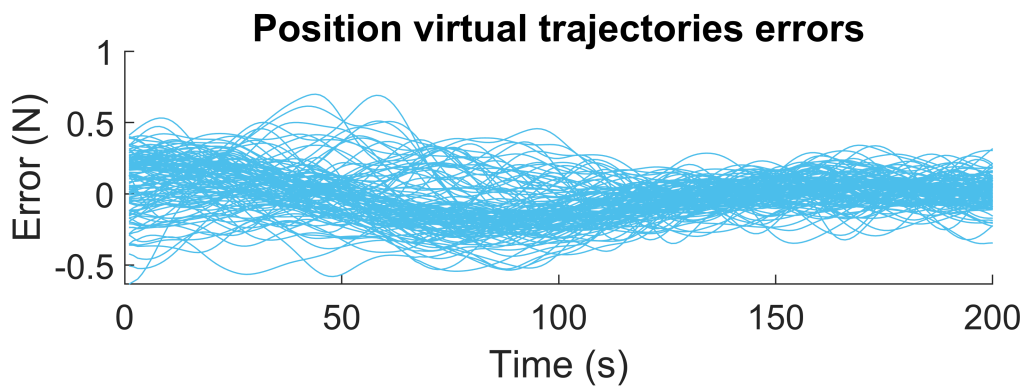
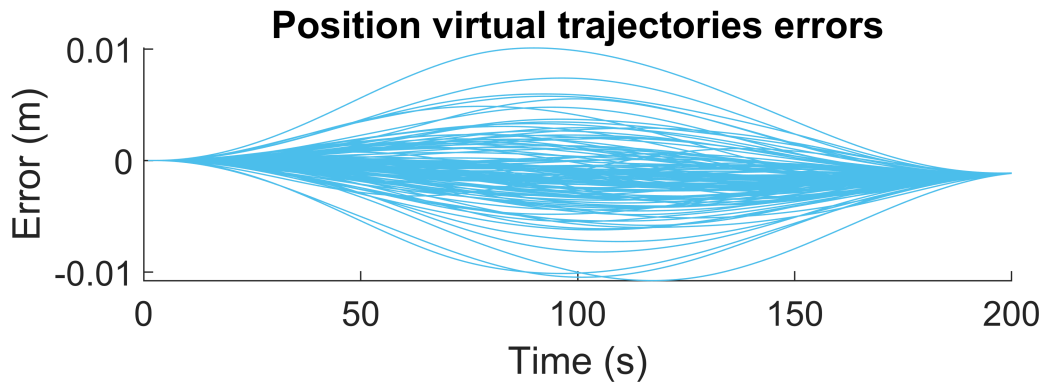
```
% Actual simulated virtual trajectories
delta_z_sim = copyObj(delta_z);
delta_fz_sim = copyObj(delta_fz);
delay_wdw = delta_z.delay;
estim_wdw = delta_z.estim_window;
for i = 1:delta_z_sim.nb_trajet
    pert_interval = delta_z_sim.pert_ind(i)+(-2:estim_wdw+1)+delay_wdw;
    delta_z_sim.virt_trajet(:,i) = z0(pert_interval);
    delta_fz_sim.virt_trajet(:,i) = fz0(pert_interval);
end
delta_z_sim.computeDiffTraject('VirtTrajMethod', 'manual');
delta_fz_sim.computeDiffTraject('VirtTrajMethod', 'manual');
% The result should be the same in this ideal scenario
```

```
%
figure('DefaultAxesFontSize',14)
subplot(2,1,1)
hold on
plot(delta_z.virt_trajet-delta_z_sim.virt_trajet, 'Color', [0.3010, 0.7450, 0.9330, 0.4])
title('Position virtual trajectories errors')
xlabel('Time (s)')
ylabel('Error (m)')
xlim([0, idx_wndw_imp_eval])
subplot(2,1,2)
hold on
plot(delta_fz.virt_trajet-delta_fz_sim.virt_trajet, 'Color', [0.3010, 0.7450, 0.9330, 0.4])
```

```

title('Position virtual trajectories errors')
xlabel('Time (s)')
ylabel('Error (N)')
xlim([0, idx_wndw_imp_eval])

```



IV-b Discrete time identification algorithms

We define two polynomials A_k and B_k such as: %% TODO rewrite this expression in accordance to the knew representation using the transfer function

$$y(t) + a_1 y(t-1) + a_0 y(t-2) = b_1 u(t) + b_0 u(t-1) + e(t)$$

$$A_k(q)y(t) = B_k(q)u(t) + e(t),$$

with q the delay operator.

```

% Data formating for identification
for i = delta_z.nb_trajet:-1:1
    data{i} = iddata(delta_z.diff_trajet(:,i), ...
        delta_fz.diff_trajet(:,i),dt);
    data{i}.TimeUnit = 's';
    data{i}.InputUnit = 'N';
    data{i}.OutputUnit = 'm';
end
data_sim = iddata(delta_z_sim.diff_trajet(:,1),...
    delta_fz_sim.diff_trajet(:,1),dt); % ideal data
data_sim.TimeUnit = 's';

```

```
data_sim.InputUnit = 'N';
data_sim.OutputUnit = 'm';
```

The ARX identification method can now be used, since we know the order of the polynomials A_k and B_k :

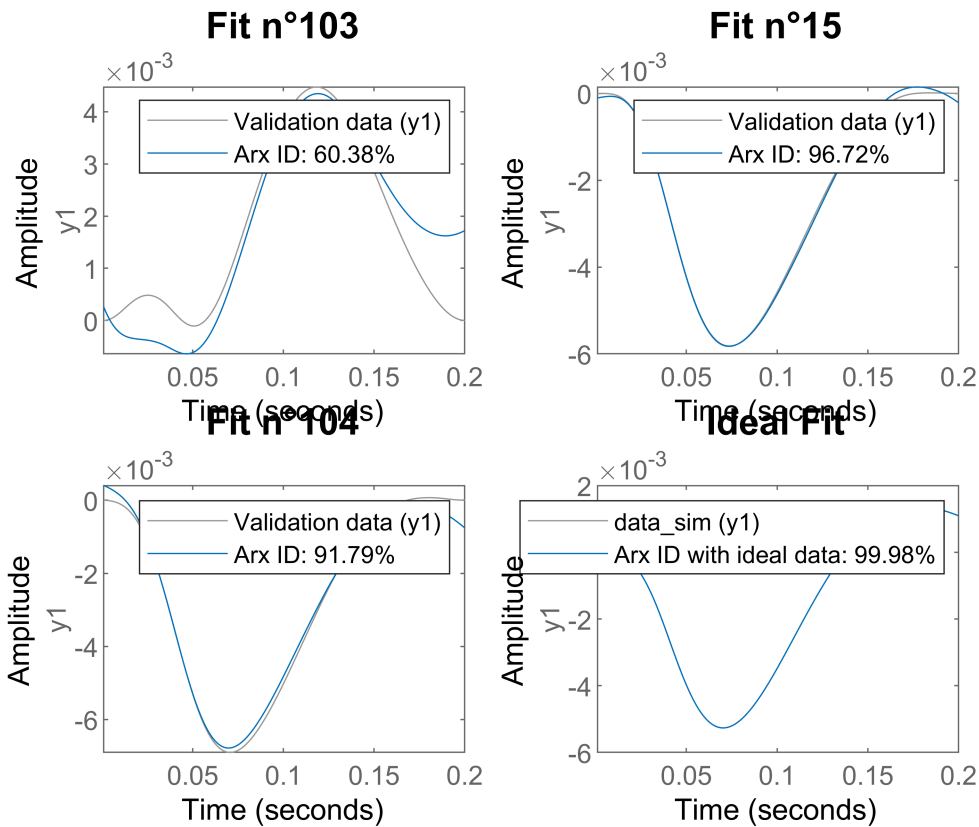
```
na = length(a_coef)-1; % order of Ak
nb = length(b_coef)-1; % order of Bk
nk = 0; % delay in Bk
for i = delta_z.nb_traject:-1:1
    arx_id{i} = arx(data{i},[na nb+1 nk], 'IntegrateNoise',true);
    arx_id{i}.Name = 'Arx ID';
end
arx_id_sim = arx(data_sim,[na nb+1 nk]);
arx_id_sim.Name = 'Arx ID with ideal data';
```

The quality of the identification can be estimated... to be continued...

```
%
% for i = delta_z.nb_traject:-1:1
%     pem_id{i} = pem(data{i},arx_id{i});
%     pem_id{i}.Name = 'Pem ID';
% end
```

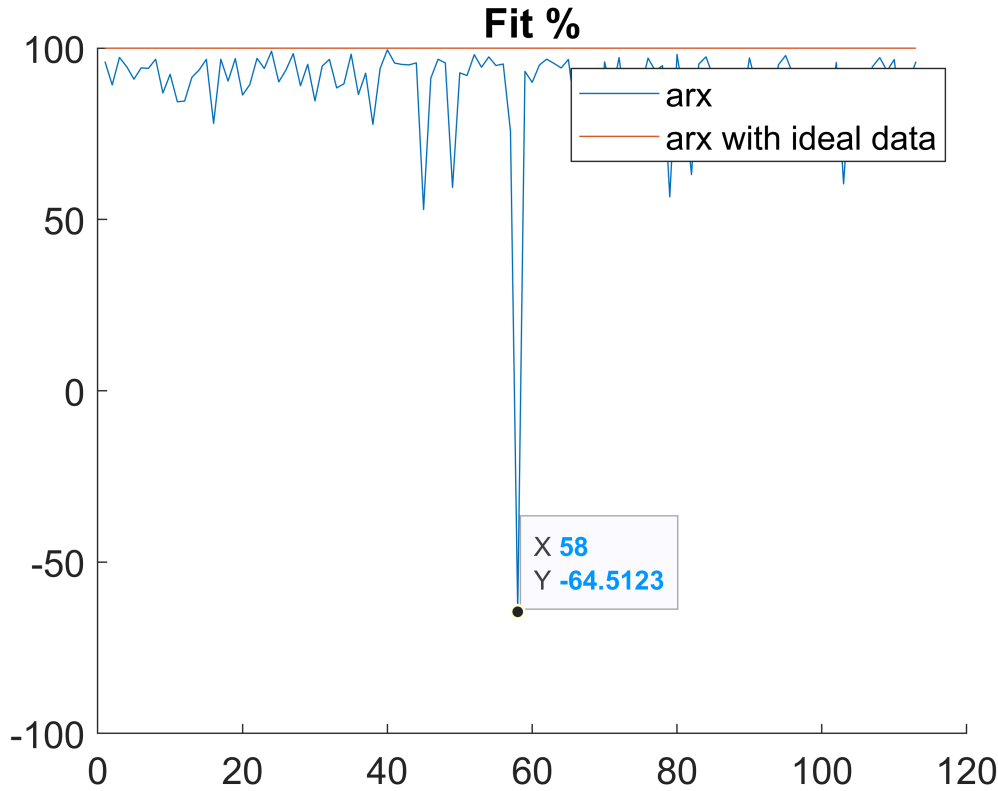
A random estimation is picked to compare the quality of the estimation:

```
rand_idx = randi([1 delta_z.nb_traject],3,1);
figure('DefaultAxesFontSize',12)
for i = 1:3
    subplot(2,2,i)
    compare(data{rand_idx(i)},arx_id{rand_idx(i)});
    title("Fit n°" + string(rand_idx(i)))
end
%figure('DefaultAxesFontSize',14)
subplot(2,2,4)
compare(data_sim,arx_id_sim);
title("Ideal Fit")
```



A wider overview is then displayed:

```
for i = delta_z.nb_trajet:-1:1
    % Report fit default horizon is set to 1, how do we change it ?
    %arx_id_fit1(i) = arx_id{i}.Report.Fit.FitPercent;
    [~,arx_id_fit(i),~] = compare(data{i},arx_id{i});
end
[~,arx_id_sim_fit,~] = compare(data_sim,arx_id_sim);
figure('DefaultAxesFontSize',14)
hold on
plot(arx_id_fit)
plot([1, delta_z.nb_trajet], [arx_id_sim_fit, arx_id_sim_fit])
title('Fit %')
legend('arx', 'arx with ideal data')
```



IV-c Impedance parameters identification

1) Numerical methodology

Once the coefficients of A_k and B_k are identified, the impedance parameters can be deduced thanks to the relations detailed in the *Discretization* paragraph for every identification conducted for each perturbation:

$$a_1 = \frac{a_1'}{a_2'}, a_0 = \frac{a_0'}{a_2'}, K_h(1 + b_0) = \frac{b_1' + b_0'}{a_2'}$$

```
% K D M identification
for i = 1:length(arx_id)
    % declare the equations system
    eq.A(1) = a1 - arx_id{i}.A(2)/arx_id{i}.A(1) == 0;
    eq.A(2) = a0 - arx_id{i}.A(3)/arx_id{i}.A(1) == 0;
    eq.B(1) = Kh*(b0 + 1) - (arx_id{i}.B(1) + arx_id{i}.B(2))/arx_id{i}.A(1) == 0;
    imp_param_id(i) = vpasolve([eq.A(1),eq.A(2),eq.B(1)],[Ks;Bs;Ms]);
    if i == 1
        warning('off','all'); % to avoid spam warnings in the loop
    end
end
warning('on','all')
```

Same process on the ideal case scenario

```

eq.A(1) = a1 - arx_id_sim.A(2)/arx_id_sim.A(1) == 0;
eq.A(2) = a0 - arx_id_sim.A(3)/arx_id_sim.A(1) == 0;
eq.B(1) = Kh - arx_id_sim.B(1)/arx_id_sim.A(1) == 0;
eq.B(2) = b0 - arx_id_sim.B(2)/arx_id_sim.B(1) == 0;
eq.B(3) = Kh*b0 - arx_id_sim.B(2)/arx_id_sim.A(1) == 0;
eq.B(4) = Kh*(b0 + 1) - (arx_id_sim.B(1) + arx_id_sim.B(2))/arx_id_sim.A(1) == 0;
imp_param_id_sim = vpasolve([eq.A(1),eq.A(2),eq.B(4)],[Ks;Bs;Ms]);
% expected arx param
double(subs(a1, [Ks, Bs, Ms], [Kv, Bv, Mv]))

```

```
ans = -1.9743
```

```
double(subs(a0, [Ks, Bs, Ms], [Kv, Bv, Mv]))
```

```
ans = 0.9753
```

```
double(subs(Kh*b0, [Ks, Bs, Ms], [Kv, Bv, Mv]))
```

```
ans = 2.4585e-06
```

2) Analytical methodology

We can also use the eigenvalues of A_d , to express the impedance parameters in terms of arx discrete coefficients. Since the poles of H_d are equal to the eigenvalues of A_d , we have the following equality:

$$\alpha_1 = \frac{-a_1 + (a_1^2 - 4a_0)^{\frac{1}{2}}}{2} = e^{\frac{-B + (B^2 - 4MK)^{\frac{1}{2}}}{2M}T} \quad (\text{iv-c-2-1})$$

$$\alpha_2 = \frac{-a_1 - (a_1^2 - 4a_0)^{\frac{1}{2}}}{2} = e^{\frac{-B - (B^2 - 4MK)^{\frac{1}{2}}}{2M}T}$$

Then if we apply the final value theorem, we can determine the gain of the transfer function and underline a final relation:

$$\lim_{t \rightarrow +\infty} f_e(t) = \lim_{s \rightarrow 0} sF_e(s) = \lim_{z \rightarrow 1} (z - 1)F_e(z)$$

Considering a unitary step input X_e :

$$\lim_{s \rightarrow 0} sF_e(s) = \lim_{s \rightarrow 0} s(Ms^2 + Bs + K)X_e(p) = K$$

$$\lim_{z \rightarrow 1} (z - 1)F_e(z) = \lim_{z \rightarrow 1} \frac{(z - 1)}{K_H} \frac{z^2 + a_1z + a_0}{z + b_0} X_e(z) = \frac{1 + a_1 + a_0}{K_H(1 + b_0)}$$

$$\Rightarrow K = \frac{1 + a_1 + a_0}{K_H(1 + b_0)}$$

The equations (iv-c-2-1) can be sum up after using the natural logarithm to find a relation between B and M :

$$\ln(\alpha_1) + \ln(\alpha_2) = -\frac{B}{M}T$$

$$\Rightarrow B = -\ln(\alpha_1 \alpha_2) \frac{M}{T} = -\ln(a_0) \frac{M}{T}$$

Finally, the expression of B and K can be injected in one of the equations (iv-c-2-1) to solve M :

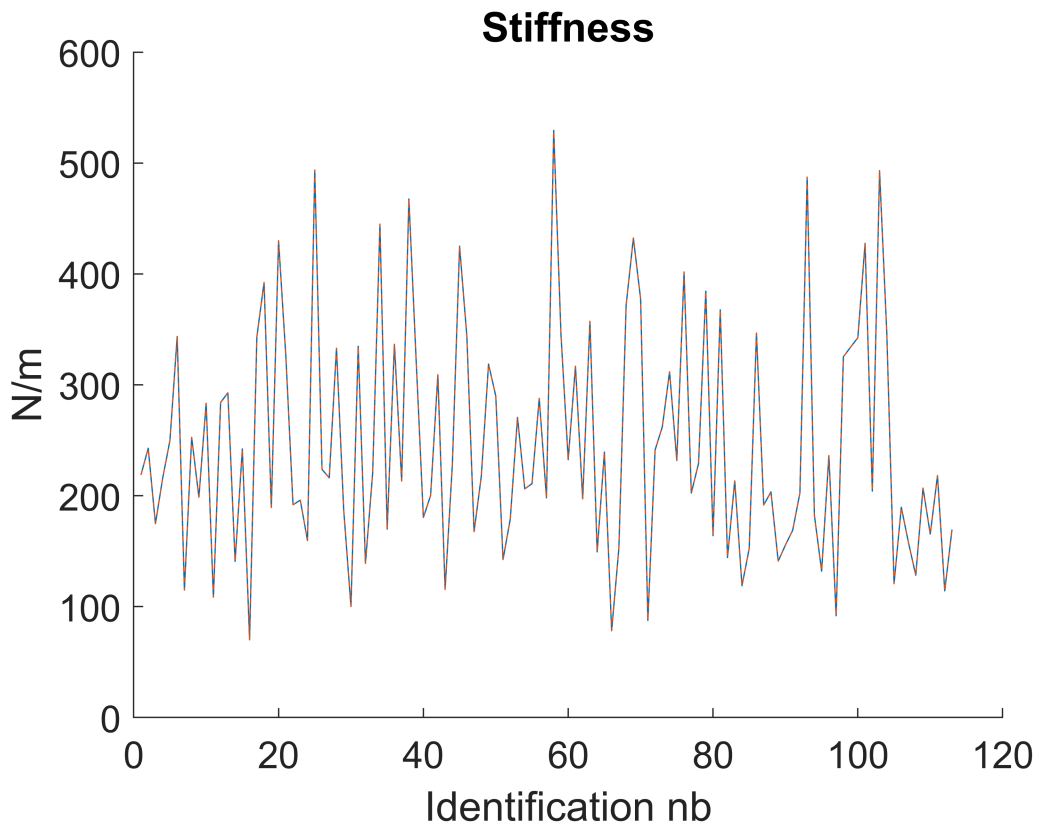
$$M = - \frac{KT^2}{\left(\ln(\alpha_1) - \frac{\ln(a_0)}{2}\right)^2 - \left(\frac{\ln(a_0)}{2}\right)^2} = \frac{-KT^2}{\ln(\alpha_1)(\ln(\alpha_1) - \ln(a_0))}$$

```
for i = length(afx_id):-1:1
    K_id(i) = (afx_id{i}.A(1) + afx_id{i}.A(2) + afx_id{i}.A(3))/...
        (afx_id{i}.B(1) + afx_id{i}.B(2));
    % if afx_id{i}.A(1) ~= 1 this will bring a wrong result
    alpha1 = (-afx_id{i}.A(2)+sqrt(afx_id{i}.A(2)^2-4*afx_id{i}.A(3)))/2;
    M_id(i) = -K_id(i)*dt^2/(log(alpha1)*(log(alpha1)-log(afx_id{i}.A(3))));
    B_id(i) = -M_id(i)/dt*log(afx_id{i}.A(3)/afx_id{i}.A(1));
end
% Ideal case scenario
K_id_sim = (afx_id_sim.A(1) + afx_id_sim.A(2) + afx_id_sim.A(3))/...
    (afx_id_sim.B(1) + afx_id_sim.B(2));
% if afx_id{i}.A(1) ~= 1 this will bring a wrong result
alpha1 = (-afx_id_sim.A(2)+sqrt(afx_id_sim.A(2)^2-4*afx_id_sim.A(3)))/2;
M_id_sim = -K_id_sim*dt^2/(log(alpha1)*(log(alpha1)-log(afx_id_sim.A(3))));
B_id_sim = -M_id_sim/dt*log(afx_id_sim.A(3)/afx_id_sim.A(1));
```

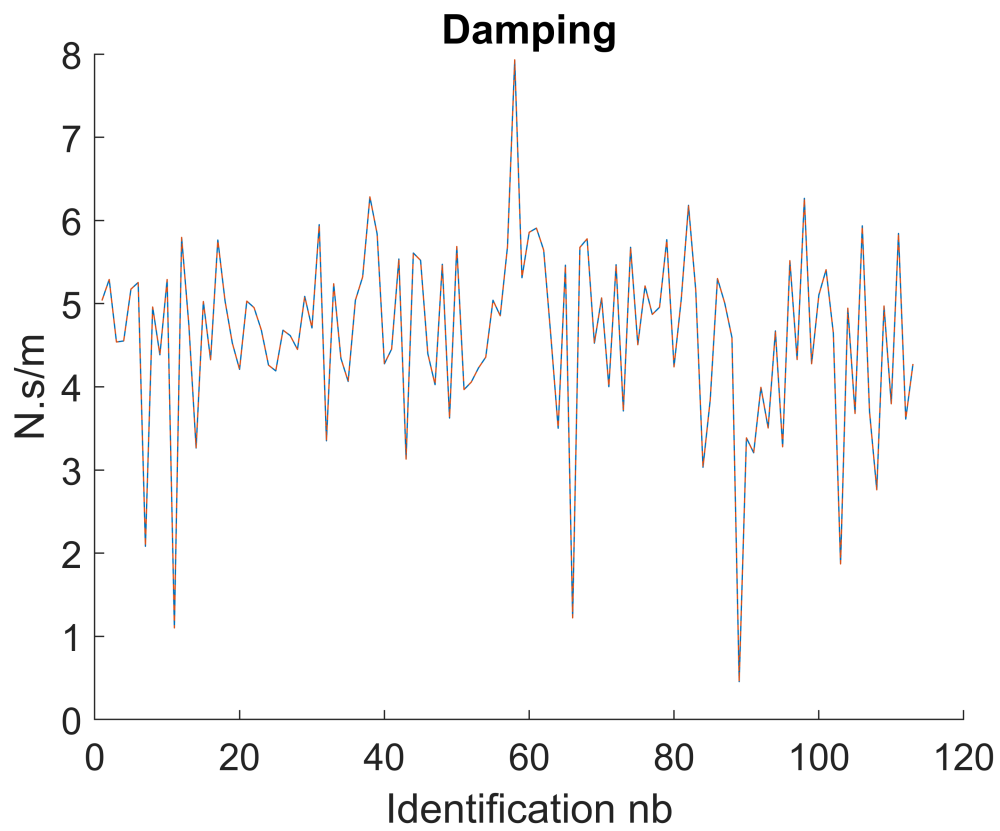
V Results

The identified parameters are:

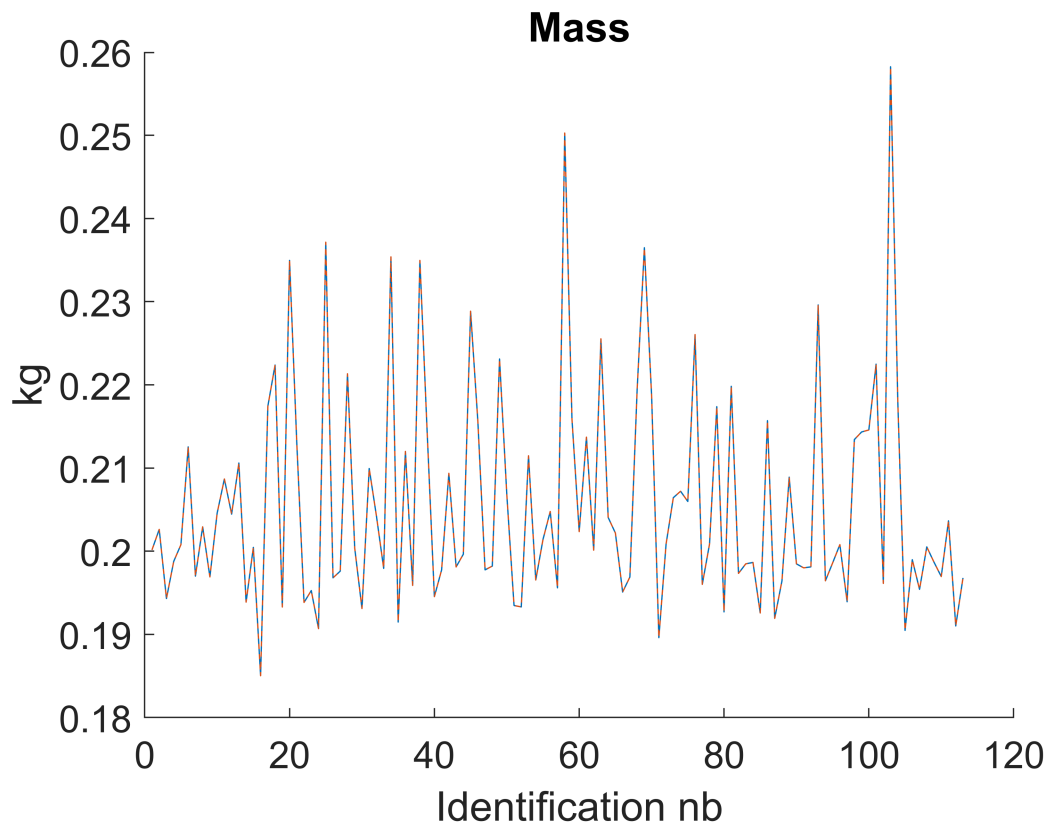
```
figure('DefaultAxesFontSize',14)
title('Stiffness')
hold on
plot([imp_param_id(:).Ks])
plot(K_id, '--')
xlabel('Identification nb')
ylabel('N/m')
```



```
figure('DefaultAxesFontSize',14)
title('Damping')
hold on
plot([imp_param_id(:).Bs])
plot(B_id, '--')
xlabel('Identification nb')
ylabel('N.s/m')
```



```
figure('DefaultAxesFontSize',14)
title('Mass')
hold on
plot([imp_param_id(:).Ms])
plot(M_id, '--')
xlabel('Identification nb')
ylabel('kg')
```



If we take away the outliers, and compute the average and the standard deviation, we have the following results:

```
mean(rmoutliers(double([imp_param_id(:).Ks])), mean(rmoutliers(K_id)))
```

```
ans = 244.6734
ans = 244.6734
```

```
mean(rmoutliers(double([imp_param_id(:).Bs])), mean(rmoutliers(B_id)))
```

```
ans = 4.7251
ans = 4.7251
```

```
mean(rmoutliers(double([imp_param_id(:).Ms])), mean(rmoutliers(M_id)))
```

```
ans = 0.2029
ans = 0.2029
```

In the ideal case scenario, where the virtual trajectories are known, the methodology gives the following relative error to the 3 impedance parameters:

```
abs(double(imp_param_id_sim.Ks)-Kv)/Kv*100, abs((K_id_sim)-Kv)/Kv*100
```

```
ans = 0.0295
ans = 0.0295
```

```
abs(double(imp_param_id_sim.Bs)-Bv)/Bv*100, abs((B_id_sim)-Bv)/Bv*100
```

```
ans = 0.0034
ans = 0.0034
```

```
abs(double(imp_param_id_sim.Ms)-Mv)/Mv*100, abs((M_id_sim)-Mv)/Mv*100
```

```
ans = 0.0563  
ans = 0.0563
```

Annexes

Functions

```
function b = copyObj(a)  
    b = eval(class(a)); %create default object of the same class as a. one valid use of eval  
    for p = properties(a).' %copy all public properties  
        try %may fail if property is read-only  
            b.(p{1}) = a.(p{1});  
        catch  
            warning('failed to copy property: %s', p);  
        end  
    end  
end
```